



Vehicle Registration System

Java Console Application — Student Project Guide

Step-by-Step Tasks with Full Requirements

Level	Intermediate Java
Tasks	10 Tasks
Topics	6 Java Concepts



Project Overview

Students will build a Vehicle Registration System as a console-based Java application. The system simulates a real-world DMV (Department of Motor Vehicles) where vehicles can be registered, searched, updated, and reported on. The project is divided into 10 progressive tasks — each one building on the previous — ensuring students learn each Java concept through hands-on practice.



Learning Objectives

- Apply Java basics: variables, loops, conditionals, input/output
- Design classes using OOP principles: encapsulation, inheritance, polymorphism, abstraction
- Use Java Collections: ArrayList, HashMap, HashSet, LinkedList
- Handle errors gracefully using custom and built-in exceptions
- Implement equals() and hashCode() correctly for object identity
- Use Java 8 Streams for filtering, sorting, grouping, and reporting



System Features (Full Application)

Feature	Description
Register Vehicle	Add a new vehicle with plate number, owner, type, and year
Search Vehicle	Find a vehicle by plate number
Update Vehicle	Update owner name or vehicle status
Delete Vehicle	Remove a vehicle from the registry
List All Vehicles	Show all registered vehicles in a table
Filter by Type	Show only cars, trucks, motorcycles, etc.
Owner History	Get all vehicles owned by one person
Expired Registrations	Find vehicles registered over 5 years ago
Statistics Report	Count by type, average year, newest/oldest
Duplicate Detection	Prevent same plate number from being registered twice

Recommended Project Structure

Create the following package structure in your IDE (IntelliJ / Eclipse / VS Code):

```
vehicle.registration/  
├── model/  
│   ├── Vehicle.java           ← Base class  
│   ├── Car.java               ← Extends Vehicle  
│   ├── Truck.java             ← Extends Vehicle  
│   └── Motorcycle.java        ← Extends Vehicle  
├── exception/  
│   ├── DuplicatePlateException.java  
│   └── VehicleNotFoundException.java  
├── service/  
│   └── RegistrationService.java ← All business logic  
├── util/  
│   └── InputValidator.java     ← Input validation helpers  
└── Main.java                  ← Entry point / Console menu
```

Task 1: Create the Vehicle Model Class

Concepts Applied: *Java Basics, OOP — Encapsulation*

What You Will Build

Create a `Vehicle.java` class that represents a vehicle in the system. This is your data model — the foundation of the entire project.

Step-by-Step Instructions

1. Create a new Java class named `Vehicle` in the model package.
2. Declare the following private fields:
 - `String plateNumber` → the unique license plate (e.g., "ABC-1234")
 - `String ownerName` → full name of the vehicle owner
 - `String vehicleType` → type: "Car", "Truck", or "Motorcycle"
 - `int registrationYear` → year the vehicle was registered (e.g., 2019)
 - `String status` → "ACTIVE" or "EXPIRED"
3. Generate a constructor that accepts all 5 fields as parameters.
4. Generate getters for all fields (use your IDE: `Alt+Insert` → `Getter`).
5. Generate a setter **ONLY** for `ownerName` and `status` (the plate number should never change).
6. Override `toString()` to return a readable one-line summary of the vehicle.

Expected `toString()` Output Format

```
[ABC-1234] | Car | Owner: Ahmed Ali | Year: 2021 | Status: ACTIVE
```

Acceptance Criteria

- Fields are private, accessed only through getters/setters
- No public fields allowed
- `toString()` returns the exact format above

 **Key Concept:** This task applies encapsulation — hiding internal data and exposing only what is needed through controlled methods.

Task 2: Build the Vehicle Hierarchy (Inheritance)

Concepts Applied: *OOP — Inheritance, Abstract Classes, Polymorphism*

What You Will Build

Make `Vehicle` an abstract class, then create three subclasses: `Car`, `Truck`, and `Motorcycle` — each with their own extra properties and behavior.

Step-by-Step Instructions

7. Convert Vehicle.java to an abstract class by adding the abstract keyword.
8. Add an abstract method: public abstract String getRegistrationLabel();
9. Create class Car extends Vehicle with an extra field: int numberOfDoors (default 4).
10. Create class Truck extends Vehicle with extra field: double cargoCapacityTons.
11. Create class Motorcycle extends Vehicle with extra field: String engineType ("Sport", "Cruiser", "Off-Road").
12. Each subclass must call super(...) in its constructor to pass data to Vehicle.
13. Each subclass must implement getRegistrationLabel() with a unique message:
 - Car: "Passenger Car — Doors: 4"
 - Truck: "Commercial Truck — Cargo: 5.0 tons"
 - Motorcycle: "Motorcycle — Engine: Sport"
14. Override toString() in each subclass to call super.toString() + add their extra info.

Test in Main.java

```
Vehicle v1 = new Car("ABC-001", "Sara Ahmed", 2022, "ACTIVE", 4);
Vehicle v2 = new Truck("TRK-099", "Ali Hassan", 2020, "ACTIVE", 8.5);
Vehicle v3 = new Motorcycle("MOT-555", "Omar Said", 2023, "ACTIVE", "Sport");
System.out.println(v1.getRegistrationLabel());
System.out.println(v2.getRegistrationLabel());
```

 **Key Concept:** Polymorphism lets you call getRegistrationLabel() on a Vehicle reference and get the right result for Car, Truck, or Motorcycle automatically.

Task 3: Implement equals() and hashCode()

Concepts Applied: equals() and hashCode() Contract

What You Will Build

Two vehicles with the same plate number should be considered equal. Implement equals() and hashCode() in Vehicle.java to enforce this rule.

Step-by-Step Instructions

15. In Vehicle.java, override the equals(Object o) method.
16. Two vehicles are equal if and only if their plateNumber is equal (case-insensitive).
17. Override hashCode() to return the hash of the plateNumber in lowercase.
18. Follow the full contract:
 - Check if o == this → return true
 - Check if o == null or class is different → return false
 - Cast o to Vehicle and compare plateNumber.equalsIgnoreCase(...)

Implementation Template

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
```

```

        if (o == null || getClass() != o.getClass()) return false;
        Vehicle vehicle = (Vehicle) o;
        return plateNumber.equalsIgnoreCase(vehicle.plateNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(plateNumber.toLowerCase());
    }

```

Test Cases — Write These in Main.java

```

Vehicle a = new Car("abc-001", "Ali", 2021, "ACTIVE", 4);
Vehicle b = new Car("ABC-001", "Sara", 2022, "ACTIVE", 2);
System.out.println(a.equals(b)); // Expected: true
System.out.println(a.hashCode() == b.hashCode()); // Expected: true

```

⚠ Important Rule: If `equals()` returns true for two objects, their `hashCode()` MUST return the same value. Breaking this rule causes bugs in `HashMap` and `HashSet`.

Task 4: Create Custom Exceptions

Concepts Applied: *Exception Handling — Custom Exceptions, throw, try/catch*

What You Will Build

Create two custom exception classes to handle specific system errors properly, instead of using generic exceptions.

Step-by-Step Instructions

Create `DuplicatePlateException.java` in the exception package:

19. Extend `RuntimeException`.
20. Constructor accepts a `String` `plateNumber`.
21. Pass message: "Plate number already registered: " + `plateNumber` to `super()`.

Create `VehicleNotFoundException.java` in the exception package:

22. Extend `RuntimeException`.
23. Constructor accepts a `String` `plateNumber`.
24. Pass message: "No vehicle found with plate: " + `plateNumber` to `super()`.

Template for `DuplicatePlateException`

```

public class DuplicatePlateException extends RuntimeException {
    public DuplicatePlateException(String plateNumber) {
        super("Plate number already registered: " + plateNumber);
    }
}

```

```
}
```

Where They Will Be Used (Next Tasks)

- Throw `DuplicatePlateException` when registering a plate that already exists
- Throw `VehicleNotFoundException` when searching for a plate that doesn't exist
- Catch both in the Main menu and print a user-friendly error message

 **Key Concept:** Custom exceptions make error messages meaningful. Instead of 'NullPointerException', the user sees 'No vehicle found with plate: XYZ-999'.

Task 5: Build the Registration Service (Collections)

Concepts Applied: *Collections — ArrayList, HashMap, HashSet*

What You Will Build

Create the RegistrationService class that stores all vehicles and provides methods to add, find, update, and delete vehicles using Java Collections.

Step-by-Step Instructions

25. Create RegistrationService.java in the service package.
26. Declare the following fields:
 - `private List<Vehicle> vehicleList = new ArrayList<>();` ← all vehicles in order
 - `private Map<String, Vehicle> plateIndex = new HashMap<>();` ← fast lookup by plate
 - `private Set<String> registeredPlates = new HashSet<>();` ← O(1) duplicate check
27. Implement `registerVehicle(Vehicle v)`:
 - Normalize plateNumber to UPPERCASE
 - Check registeredPlates — if plate exists, throw DuplicatePlateException
 - Add to all three collections
28. Implement `findByPlate(String plate)`:
 - Normalize plate to UPPERCASE
 - If not in plateIndex, throw VehicleNotFoundException
 - Return the Vehicle object
29. Implement `deleteVehicle(String plate)`:
 - Find vehicle (reuse `findByPlate` — exception handles missing case)
 - Remove from all three collections
30. Implement `updateOwner(String plate, String newOwner)` — find vehicle, call `setOwnerName()`
31. Implement `getAllVehicles()` — return `Collections.unmodifiableList(vehicleList)`

 **Why Three Collections?** ArrayList preserves insertion order. HashMap enables O(1) lookup by plate. HashSet enables O(1) duplicate detection. Each serves a different purpose.

Task 6: Build the Console Menu (Java Basics)

Concepts Applied: *Java Basics — Scanner, loops, switch, input validation*

What You Will Build

Create the Main.java entry point with a text-based console menu. The user should be able to interact with the system by typing a number and pressing Enter.

Expected Console Output

```
=====
VEHICLE REGISTRATION SYSTEM  v1.0
```

```
=====
1. Register New Vehicle
2. Search Vehicle by Plate
3. Update Owner Name
4. Delete Vehicle
5. List All Vehicles
6. Filter by Vehicle Type
7. Show Owner History
8. Show Expired Registrations
9. Statistics Report
0. Exit
=====
Enter your choice: _
```

Step-by-Step Instructions

32. Create Main.java with a main() method.
33. Create a Scanner `sc = new Scanner(System.in);`
34. Create a RegistrationService `service = new RegistrationService();`
35. Use a `while(true)` loop to keep the menu running until user selects 0.
36. Print the menu, then read the user's choice with `sc.nextInt()`.
37. Use a switch statement to call the right service method.
38. For option 1 (Register), prompt for: plate number, owner name, type, year.
39. Wrap ALL service calls in try/catch blocks — catch your custom exceptions and print the message.
40. Call `sc.nextLine()` after `sc.nextInt()` to consume the newline character.

Handle Bad Input

- Wrap `sc.nextInt()` in try/catch for `InputMismatchException`
- Print: "Invalid input. Please enter a number." on error
- Call `sc.nextLine()` inside catch to clear the bad input

 **Common Mistake:** Forgetting `sc.nextLine()` after `sc.nextInt()` causes the next String input to be skipped. Always clear the buffer.

Task 7: Filter & Search with Java 8 Streams

Concepts Applied: *Java 8 Streams — filter, map, sorted, collect*

What You Will Build

Add stream-based search and filter methods to RegistrationService. No loops — use the Stream API exclusively.

Methods to Implement

Method 1: filterByType(String type)

Return a List<Vehicle> of all vehicles matching the given type (case-insensitive).

```
return vehicleList.stream()
    .filter(v -> v.getVehicleType().equalsIgnoreCase(type))
    .collect(Collectors.toList());
```

Method 2: getVehiclesByOwner(String ownerName)

Return all vehicles owned by a specific person (case-insensitive partial match).

```
return vehicleList.stream()
    .filter(v -> v.getOwnerName().toLowerCase()
        .contains(ownerName.toLowerCase()))
    .collect(Collectors.toList());
```

Method 3: getExpiredRegistrations(int currentYear)

Return all vehicles registered more than 5 years ago (relative to currentYear).

```
return vehicleList.stream()
    .filter(v -> (currentYear - v.getRegistrationYear()) > 5)
    .sorted(Comparator.comparingInt(Vehicle::getRegistrationYear))
    .collect(Collectors.toList());
```

Method 4: getSortedByYear(boolean ascending)

Return all vehicles sorted by registration year.

```
Comparator<Vehicle> comp =
    Comparator.comparingInt(Vehicle::getRegistrationYear);
if (!ascending) comp = comp.reversed();
return vehicleList.stream().sorted(comp).collect(Collectors.toList());
```

 **Key Concept:** Streams are lazy — no processing happens until a terminal operation like collect() or forEach() is called. Each intermediate operation (filter, map, sorted) builds a pipeline.

Task 8: Statistics Report with Streams

Concepts Applied: *Java 8 Streams — groupingBy, counting, mapToInt, IntSummaryStatistics*

What You Will Build

Add a `printStatistics()` method that uses advanced Stream operations to produce a complete statistical report printed to the console.

Report Format to Produce

```
===== REGISTRATION STATISTICS =====
Total Vehicles      : 12
Average Year        : 2020
Newest Vehicle       : 2024
Oldest Vehicle       : 2015
-----
Vehicles by Type:
  Car                : 7
  Truck              : 3
  Motorcycle          : 2
-----
ACTIVE vehicles : 10
EXPIRED vehicles: 2
=====
```

Step-by-Step Instructions

41. Use `IntSummaryStatistics` to get count, average, min, max in one stream operation:

```
IntSummaryStatistics stats = vehicleList.stream()
    .mapToInt(Vehicle::getRegistrationYear)
    .summaryStatistics();
```

42. Use `Collectors.groupingBy` + `Collectors.counting()` to count by type:

```
Map<String, Long> byType = vehicleList.stream()
    .collect(Collectors.groupingBy(Vehicle::getVehicleType,
        Collectors.counting()));
```

43. Use `Collectors.partitioningBy` to split ACTIVE vs EXPIRED:

```
Map<Boolean, Long> byStatus = vehicleList.stream()
    .collect(Collectors.partitioningBy(
        v -> v.getStatus().equals("ACTIVE"),
        Collectors.counting()));
```

44. Print all results in the format shown above using `System.out.printf()`.

 **Pro Tip:** `IntSummaryStatistics` captures 5 things at once: count, sum, min, max, average. Far more efficient than 5 separate stream operations.

Task 9: Input Validation (Exception Handling — Advanced)

Concepts Applied: *Exception Handling — checked exceptions, validation, custom messages*

What You Will Build

Create an InputValidator utility class that validates all user input before it reaches the service layer. Throw meaningful exceptions when input is invalid.

Validation Rules

Field	Rule	Error Message
plateNumber	Not null, not empty, matches [A-Z0-9-]{3,10}	"Plate number must be 3-10 alphanumeric characters"
ownerName	Not null, not empty, min 3 chars, letters only	"Owner name must contain only letters"
vehicleType	Must be Car, Truck, or Motorcycle (case-insensitive)	"Invalid type. Use: Car, Truck, Motorcycle"
registrationYear	Between 1990 and current year	"Year must be between 1990 and " + currentYear

Step-by-Step Instructions

45. Create class InputValidator with all static methods.
46. Create a new exception: InvalidInputException extends IllegalArgumentException.
47. Each validation method should throw InvalidInputException with a clear message if the rule fails.
48. Call validators from Main.java before passing data to the service.
49. In Main.java, catch InvalidInputException and print: "Input Error: " + e.getMessage().

 **Key Concept:** Validation at the boundary (before the service layer) keeps business logic clean. Never let invalid data reach your core objects.

Task 10: Final Integration — Complete Console App

Concepts Applied: All Concepts — Integration, Testing, Streams, Exceptions

What You Will Build

Put everything together. Seed the system with sample data, run through all 9 menu options, and verify the output is correct. This is your final deliverable.

Pre-load Sample Data (Seed Data)

Add this to the top of main() before showing the menu:

```
service.registerVehicle(new Car("CAR-001", "Ahmed Ali", 2019, "EXPIRED", 4));
service.registerVehicle(new Car("CAR-002", "Sara Kamel", 2023, "ACTIVE", 2));
service.registerVehicle(new Truck("TRK-001", "Mohamed Said", 2021, "ACTIVE",
10.0));
service.registerVehicle(new Truck("TRK-002", "Laila Nour", 2017, "EXPIRED",
5.5));
service.registerVehicle(new Motorcycle("MOT-001", "Omar Fathi", 2022, "ACTIVE",
"Sport"));
service.registerVehicle(new Motorcycle("MOT-002", "Nadia Hamed", 2024, "ACTIVE",
"Cruiser"));
```

Final Checklist — All 10 Items Must Work

#	Test Case	Pass?
1	Register a new car — it appears in list	☐
2	Try to register the same plate twice — error message shown	☐
3	Search for plate CAR-001 — correct vehicle shown	☐
4	Search for plate ZZZ-999 (not found) — error shown	☐
5	Update owner of TRK-001 — change reflected in list	☐
6	Delete MOT-002 — no longer appears in list	☐
7	Filter by type 'Truck' — only trucks shown	☐
8	Show expired registrations — CAR-001 and TRK-002 appear	☐
9	Statistics report — counts, average year, breakdown correct	☐
10	Enter invalid choice '99' — invalid input message shown	☐



Concept Reference Summary

Quick reference showing which task covers which Java concept.

Java Concept	Key Classes / Methods Used	Tasks
Java Basics	Scanner, while, switch, System.out.printf	Task 1, Task 6
OOP — Encapsulation	private fields, getters, setters, toString()	Task 1
OOP — Inheritance	abstract class, extends, super(), @Override	Task 2
OOP — Polymorphism	abstract method, Vehicle reference → subtype	Task 2
equals() & hashCode()	Objects.hash(), equalsIgnoreCase()	Task 3
Custom Exceptions	extends RuntimeException, throw, try/catch	Task 4, Task 9
Collections — List	ArrayList, add, remove, unmodifiableList	Task 5
Collections — Map	HashMap, put, get, remove	Task 5
Collections — Set	HashSet, contains, add	Task 5
Streams — filter/collect	stream().filter().collect(toList())	Task 7
Streams — sorting	sorted(Comparator.comparingInt(...))	Task 7
Streams — groupingBy	Collectors.groupingBy, counting()	Task 8
Streams — statistics	mapToInt().summaryStatistics()	Task 8
Input Validation	regex matches(), IllegalArgumentException	Task 9



Grading Suggestion

Criteria	Points	Score
Vehicle class with correct encapsulation (Task 1)	10	
Inheritance hierarchy compiles and runs (Task 2)	15	
equals() and hashCode() correct behavior (Task 3)	10	
Custom exceptions thrown in correct places (Task 4)	10	
Collections — all 3 used correctly (Task 5)	15	
Console menu with all 9 options working (Task 6)	10	
Stream filter / sort methods working (Task 7)	15	

Statistics report with groupingBy (Task 8)	10	
Input validation with exceptions (Task 9)	5	
Code quality: naming, comments, no code duplication	10	